

QCM SQL Data Analyst

20 questions notées /20



Avant ton test technique. Dialecte PostgreSQL · 4 paliers progressifs · corrigé avec mécanisme et erreur type.

Ta requête tourne ne veut pas dire que tu as compris le mécanisme. Chez Datafuji, on a vu un candidat présenter un chiffre d'affaires trois fois trop élevé sans s'en apercevoir, et un autre dont le filtre censé exclure une liste ne renvoyait plus aucune ligne, sans qu'aucun des deux ne sache pourquoi sur le moment.

Toutes les requêtes de ce test sont écrites pour PostgreSQL, pour éviter les comportements ambigus d'un moteur à l'autre (GROUP BY strict, frame de fenêtrage explicite). Objectif : repérer les 2 ou 3 blocs, jointures, agrégations, sous-requêtes ou fenêtrage, où revenir avant l'entretien.

Repères : 12/20 = junior, 17/20 = profil qu'on garde.

Candidat _____ Date _____ Note _____ /20

Bases : lire une requête avant de la corriger.

Cinq questions sur les jointures. Toutes les requêtes de ce test sont écrites pour PostgreSQL.

Question 1. Tu dois lister tous les clients, y compris ceux qui n'ont jamais passé de commande, avec leurs éventuelles commandes associées. Quelle jointure utiliser entre `clients` et `commandes` ?

- a) `FROM clients c INNER JOIN commandes cmd ON c.id = cmd.client_id`
- b) Un `CROSS JOIN` entre `clients` et `commandes`, puis un filtre en `WHERE`.
- c) `FROM clients c RIGHT JOIN commandes cmd ON c.id = cmd.client_id`
- d) `FROM clients c LEFT JOIN commandes cmd ON c.id = cmd.client_id`

Question 2. La table `employees(id, nom, manager_id)` contient à la fois les managers et les employés. Quelle requête ramène chaque employé avec le nom de son manager ?

- a) `SELECT e.nom, m.nom FROM employees e JOIN employees m ON e.manager_id = m.id`
- b) `SELECT e.nom, m.nom FROM employees e JOIN employees m ON e.id = m.manager_id`
- c) `SELECT nom FROM employees GROUP BY manager_id`
- d) `SELECT e.nom, e.manager_id FROM employees e JOIN employees ON e.id = e.manager_id`

Question 3. Un collègue joint `commandes` à `lignes_commande` sur `client_id` (au lieu de `commande_id`) pour calculer le total par commande. Le résultat surestime largement chaque total. Quelle est la cause ?

- a) `SUM()` ne fonctionne pas correctement avec un `JOIN`.
- b) Il manque un `ORDER BY` avant l'agrégation.
- c) `client_id` n'est pas une clé unique côté `lignes_commande` : chaque commande de ce client se retrouve dupliquée pour chaque ligne de commande de ce même client.
- d) `GROUP BY` doit précéder le `JOIN` dans l'ordre d'écriture de la requête.

Question 4. Avec `FROM clients c LEFT JOIN commandes cmd ON c.id = cmd.client_id`, tu veux garder tous les clients mais ne compter que leurs commandes livrées (`statut = 'livree'`). Où placer cette condition ?

- a) Dans le `WHERE`, après le `LEFT JOIN`.
- b) Peu importe l'emplacement, le résultat est identique.
- c) Dans une clause `HAVING`, après un `GROUP BY` ajouté pour l'occasion.
- d) Dans le `ON` du `LEFT JOIN`, aux côtés de la condition de jointure.

Question 5. Tu joins `clients`, `commandes` et `lignes_commande` pour un rapport mensuel. Après exécution, le nombre de lignes semble énorme. Quelle démarche standard pour localiser le fan-out ?

- a) Vérifier le `COUNT(*)` après chaque jointure ajoutée une à une, en le comparant au nombre de lignes attendu à ce stade.
- b) Relancer la requête plusieurs fois pour voir si le nombre change.
- c) Ajouter `DISTINCT` en fin de requête sans investiguer plus loin.
- d) Retirer une des tables jointes au hasard jusqu'à ce que le total paraisse correct.

Agrégations : le piège n'est jamais dans le GROUP BY seul.

Cinq questions sur ce que PostgreSQL accepte, refuse ou renvoie faux au moment d'agréger. La difficulté monte d'un cran.

Question 6. Tu veux ne garder que les villes ayant plus de 10 clients. Quelle clause filtre sur `COUNT(*)` ?

- a) `WHERE COUNT(*) > 10`
- b) `ORDER BY COUNT(*) > 10`
- c) `HAVING COUNT(*) > 10`
- d) `GROUP BY COUNT(*) > 10`

Question 7. La table `clients` contient 100 enregistrements, dont 12 avec `email NULL`. Parmi les 88 enregistrements restants, 5 valeurs d'email sont des doublons. Que renvoie `COUNT(DISTINCT email)` ?

- a) 100
- b) 88
- c) 12
- d) 83

Question 8. Sous PostgreSQL, cette requête est exécutée : `SELECT client_id, nom, SUM(montant) FROM commandes GROUP BY client_id`. Que se passe-t-il ?

- a) Erreur : la colonne `nom` doit apparaître dans le `GROUP BY` ou être enveloppée dans une fonction d'agrégation.
- b) La requête s'exécute normalement, `nom` prend une valeur arbitraire.
- c) La requête s'exécute mais `nom` devient toujours `NULL`.
- d) PostgreSQL ignore automatiquement la colonne `nom` et l'omet du résultat.

Question 9. Tu calcules la moyenne du prix par catégorie avec `SELECT categorie, AVG(prix) FROM produits p JOIN avis a ON a.produit_id = p.id GROUP BY categorie` (une ligne par avis client, plusieurs avis par produit). Le résultat semble faussé vers le haut pour les produits populaires. Pourquoi ?

- a) `AVG` ignore les valeurs négatives dans le calcul.
- b) `GROUP BY` doit être utilisé avant le `JOIN` pour être valide.
- c) Il manque un `DISTINCT` dans la clause `SELECT`.
- d) Le `JOIN` vers `avis` duplique chaque produit pour chaque avis reçu : les produits les plus commentés pèsent artificiellement plus lourd dans la moyenne.

Question 10. La colonne `note` contient des `NULL` pour les commandes sans avis, sur 20 lignes dont 5 `NULL`. `AVG(note)` calcule la moyenne sur combien de lignes ?

- a) 20
- b) 5
- c) 15
- d) 0

Sous-requêtes : la ligne qui sépare junior et confirmé.

Cinq questions sur `IN`, `EXISTS`, sous-requêtes corrélées et CTE, là où beaucoup de candidats hésitent en entretien.

Question 11. Tu veux lister les clients ayant au moins une commande, entre `WHERE client_id IN (SELECT client_id FROM commandes)` et `WHERE EXISTS (SELECT 1 FROM commandes WHERE commandes.client_id = clients.id)`. Quelle affirmation est vraie ?

- a) IN est toujours strictement plus rapide qu'EXISTS, quel que soit le contexte.
- b) EXISTS s'arrête à la première ligne trouvée; sa forme négative NOT EXISTS reste fiable même si la sous-requête contient des NULL, contrairement à NOT IN.
- c) Les deux syntaxes sont interdites par le standard SQL.
- d) EXISTS ne peut jamais être corrélé à la requête externe.

Question 12. La sous-requête `SELECT id_referent FROM blacklist` contient une ligne avec `id_referent NULL`. Que renvoie `WHERE client_id NOT IN (SELECT id_referent FROM blacklist)` ?

- a) Toutes les lignes où `client_id` n'est pas dans la liste.
- b) Une erreur de syntaxe, NULL n'étant pas autorisé dans une sous-requête.
- c) Uniquement les lignes où `client_id` est NULL.
- d) Aucune ligne : la comparaison avec NULL rend la condition « inconnue » pour chaque ligne évaluée.

Question 13. Tu veux comparer chaque commande à la moyenne de son propre client, dans le même SELECT. Quelle sous-requête est corrélée ?

- a) `SELECT montant, (SELECT AVG(montant) FROM commandes) FROM commandes c1`
- b) `SELECT montant, (SELECT AVG(montant) FROM commandes c2 WHERE c2.client_id = c1.client_id) FROM commandes c1`
- c) `SELECT montant FROM commandes WHERE montant > (SELECT AVG(montant) FROM commandes)`
- d) `SELECT AVG(montant) FROM commandes GROUP BY client_id`

Question 14. `WITH gros_clients AS (SELECT client_id FROM commandes GROUP BY client_id HAVING SUM(montant) > 1000) SELECT * FROM clients WHERE id IN (SELECT client_id FROM gros_clients)` donne le même résultat que quelle sous-requête imbriquée directe ?

- a) `SELECT * FROM clients WHERE id IN (SELECT client_id FROM commandes WHERE SUM(montant) > 1000)`
- b) `SELECT * FROM clients WHERE id IN (SELECT client_id FROM commandes)`
- c) `SELECT * FROM clients WHERE id IN (SELECT client_id FROM commandes GROUP BY client_id HAVING SUM(montant) > 1000)`
- d) `SELECT * FROM clients JOIN commandes ON clients.id = commandes.client_id`

Question 15. `SELECT nom, (SELECT montant FROM commandes WHERE commandes.client_id = clients.id) FROM clients`. Un client a 3 commandes. Que se passe-t-il ?

- a) Erreur : la sous-requête scalaire renvoie plus d'une ligne.
- b) La requête renvoie la première commande trouvée, sans avertissement.
- c) La requête renvoie 3 lignes pour ce client, une par commande.
- d) La requête renvoie NULL automatiquement pour ce client.

Fenêtrage : la notion qui distingue vraiment un profil junior.

Cinq questions pour clore le test sur le point le plus discriminant, celui qui distingue un profil junior d'un profil qu'on garde direct.

Question 16. Pour garder uniquement la dernière commande de chaque client, quelle requête est correcte ?

- a) `SELECT * FROM commandes WHERE ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY date DESC) = 1`
- b) `SELECT * FROM commandes GROUP BY client_id ORDER BY date DESC LIMIT 1`
- c) `SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY date DESC) AS rn FROM commandes) t WHERE rn = 1`
- d) `SELECT MAX(date) FROM commandes GROUP BY client_id`

Question 17. Trois vendeurs sont à égalité au 2^e rang de chiffre d'affaires. Quelle fonction attribue les rangs 1, 2, 2, 2, 5 (en sautant 3 et 4) ?

- a) ROW_NUMBER()
- b) DENSE_RANK()
- c) NTILE(4)
- d) RANK()

Question 18. Un collègue écrit `SELECT client_id, montant, SUM(montant) OVER (PARTITION BY client_id) AS total_client FROM commandes` en pensant obtenir une ligne par client, comme un `GROUP BY`. Que reçoit-il réellement ?

- a) Une ligne par commande, avec le total du client répété sur chaque ligne.
- b) Une ligne par client, comme un `GROUP BY` classique.
- c) Une erreur : `SUM` ne peut pas être combiné avec `OVER`.
- d) Une ligne par `client_id` distinct, triée par montant décroissant.

Question 19. Tu veux calculer la variation du chiffre d'affaires par rapport au mois précédent, ligne par ligne, sans self-join. Quelle fonction utiliser ?

- a) `RANK() OVER (ORDER BY mois)`
- b) `LAG(ca) OVER (ORDER BY mois)`
- c) `SUM(ca) OVER (ORDER BY mois)`
- d) `NTILE(12) OVER (ORDER BY mois)`

Question 20. Tu calcules un cumul avec `SUM(montant) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`. Deux commandes ont exactement la même date. Le cumul semble incohérent selon l'ordre de lecture. Pourquoi ?

- a) Il faut utiliser `RANGE` au lieu de `ROWS` dès que deux dates sont égales.
- b) `ROWS` s'appuie sur l'ordre physique des lignes, non garanti déterministe entre deux dates égales sans critère de départage dans l'`ORDER BY`.
- c) `SUM` ne peut pas être utilisé avec `ROWS BETWEEN`.
- d) `OVER` ne peut pas contenir de clause `ORDER BY`.

Corrigé groupé.

Un point par bonne réponse. Chaque explication donne le mécanisme exact et l'erreur type à ne plus refaire. Moins de 3/5 sur un palier : c'est ce bloc que tu revois en premier avant l'entretien.

1. **d)** Le `LEFT JOIN` conserve toutes les lignes de la table de gauche (clients) même sans correspondance : les colonnes de commandes passent à `NULL`. Erreur type : partir d'un `INNER JOIN` par réflexe et perdre silencieusement les clients sans commande.
2. **a)** Le self-join compare deux lignes de la même table via deux alias distincts (e et m) : l'employé rejoint son manager via `manager_id = id` de l'alias m. Erreur type : joindre sur `e.id = m.manager_id`, ce qui inverse la relation et ramène les subordonnés au lieu du manager.
3. **c)** `client_id` n'est pas une clé unique côté `lignes_commande` : chaque ligne de `lignes_commande` partageant ce client multiplie la commande d'origine (fan-out). Erreur type : joindre sur une colonne de regroupement au lieu de la vraie clé (`commande_id`), ce qui gonfle silencieusement les totaux.
4. **d)** Placée dans le `ON`, la condition sur commandes ne filtre que la jointure : les clients sans commande restent dans le résultat avec des colonnes à `NULL`. Erreur type : mettre cette condition dans le `WHERE`, ce qui élimine ces lignes `NULL` et transforme de fait le `LEFT JOIN` en `INNER JOIN`.
5. **a)** Comparer le `COUNT(*)` attendu après chaque jointure ajoutée permet de repérer immédiatement la table qui fait exploser le nombre de lignes. Erreur type : ajouter les trois tables d'un coup et ne vérifier le total qu'à la fin, quand la source du fan-out n'est plus identifiable.
6. **c)** `HAVING` filtre après le `GROUP BY`, sur le résultat déjà agrégé (ici `COUNT(*)`) ; `WHERE` ne peut pas référencer un agrégat, car il s'exécute avant le regroupement. Erreur type : écrire `WHERE COUNT(*) > 10`, ce que le moteur refuse.
7. **d)** `COUNT(DISTINCT email)` ignore les 12 `NULL` et dédouble les valeurs restantes (88 moins 5 doublons) : 83. Erreur type : confondre avec `COUNT(email)`, qui aurait renvoyé 88 sans dédoublement.

- 8. a)** Sous PostgreSQL, toute colonne du SELECT non agrégée doit figurer dans le GROUP BY : nom seul déclenche une erreur bloquante. Erreur type : croire que le moteur choisit une valeur arbitraire pour nom sans se plaindre.
- 9. d)** Chaque avis ajoute une ligne pour le même produit : un produit très commenté pèse plusieurs fois dans le calcul de la moyenne par catégorie, ce qui la tire vers le haut. Erreur type : agréger directement après un JOIN qui a changé le grain de la table, sans neutraliser les doublons avant.
- 10. c)** AVG() ne compte que les lignes non NULL : sur 20 lignes dont 5 NULL, la moyenne porte sur 15 valeurs, pas 20. Erreur type : présenter ce chiffre comme une moyenne sur l'ensemble des commandes, alors que les commandes sans avis sont silencieusement exclues.
- 11. b)** EXISTS s'arrête dès la première ligne correspondante trouvée. Le risque NULL vise sa forme négative : NOT EXISTS reste fiable avec des NULL dans la sous-requête, contrairement à NOT IN, qui peut ne renvoyer aucune ligne (piège de la question suivante). Erreur type : généraliser aux formes positives IN/EXISTS un piège qui ne concerne que leurs négations.
- 12. d)** Un NULL dans la liste de comparaison rend la condition NOT IN « inconnue » pour chaque ligne testée : la requête ne renvoie aucune ligne, même quand des correspondances existent. Erreur type : utiliser NOT IN sur une colonne pouvant contenir des NULL au lieu de NOT EXISTS, qui reste sûr dans ce cas.
- 13. b)** La sous-requête est corrélée car elle référence client_id de la ligne externe (c1) : elle est réévaluée pour chaque ligne de commandes. Erreur type : écrire une sous-requête non corrélée (moyenne globale, sans filtre sur c1), ce qui compare chaque commande à la moyenne de toute la table au lieu de celle de son client.
- 14. c)** La CTE ne fait que nommer la même logique : un GROUP BY avec HAVING SUM(montant) > 1000 imbriqué directement dans le IN produit un résultat identique. Erreur type : oublier le HAVING dans la version imbriquée et filtrer sur SUM dans un WHERE, ce qui échoue avant le regroupement.
- 15. a)** Une sous-requête utilisée comme valeur scalaire dans le SELECT doit renvoyer exactement une ligne : si le client a 3 commandes, le moteur lève une erreur de type « more than one row returned ». Erreur type : oublier une agrégation (MAX) ou un LIMIT avec ORDER BY pour ramener la sous-requête à une seule valeur.
- 16. c)** Une fonction de fenêtre comme ROW_NUMBER() est calculée après le WHERE : il faut l'isoler dans une sous-requête ou une CTE avant de filtrer rn = 1. Erreur type : écrire directement WHERE ROW_NUMBER() = 1, ce qui provoque une erreur car la fonction n'existe pas encore à ce stade de l'exécution.
- 17. d)** RANK() laisse un trou dans la numérotation après une égalité (1, 2, 2, 2, 5, en sautant 3 et 4), contrairement à DENSE_RANK() qui ne saute aucun rang. Erreur type : utiliser RANK() en pensant obtenir une suite continue, ce qui fausse un classement top N basé sur le rang.
- 18. a)** SUM() OVER (PARTITION BY ...) conserve une ligne par commande et répète le total du client sur chacune, contrairement à GROUP BY qui réduit à une ligne par groupe. Erreur type : confondre les deux et s'étonner d'obtenir plus de lignes que de clients en sortie.
- 19. b)** LAG(ca) OVER (ORDER BY mois) va chercher directement la valeur de la ligne précédente selon l'ordre défini, sans self-join ni sous-requête corrélée. Erreur type : reconstruire ce calcul avec un self-join sur mois moins un, ce qui casse dès qu'un mois manque dans la série.
- 20. b)** ROWS BETWEEN s'appuie sur l'ordre physique des lignes retournées : si deux dates sont strictement égales, l'ORDER BY seul ne garantit pas un ordre déterministe entre elles, et le cumul peut varier d'une exécution à l'autre. Erreur type : faire confiance à ROWS pour un cumul sur des dates sans ajouter de critère de départage (un id, par exemple) dans l'ORDER BY.

Fait main, en solo.

Si ce test t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

